

Data Structures 1 - Spring 2025 - Assignment A2

Group Members:

1. Weizhi, LU (999022064)
2. Lin, QIU (999016306)
3. Hongwei, GUO (999014780)

EXERCISE 1: Min-Heap [20-points]

1. Representation Invariant & Abstraction Function

Representation : $\text{Heap}\langle T \rangle = \langle A: \text{array}\langle T \rangle, \text{size: int} \rangle$

Representation Invariant :

$$\text{RI} = 0 \leq \text{size} \leq A.\text{length} \ \&\& \ \forall 1 < i \leq A.\text{length}, A[\lfloor i/2 \rfloor] \leq A[i]$$

Abstraction Function :

$$\text{Abs}(R) = \text{Abs}(R, 1), \text{ where}$$

$$\text{Abs}(R, i) = \begin{cases} \langle R.A[i], \text{Abs}(R, 2 * i), \text{Abs}(R, 2 * i + 1) \rangle, & \text{if } i \leq R.\text{size} \\ \text{Nil}, & \text{if } i > R.\text{size} \end{cases}$$

2. Check the Representation Invariant

[code in the wet part : `MinHeapRep`]

3. MIN-HEAPIFY

```
1  MIN-HEAPIFY(A:T[], size:int, i:int):
2      curr = i
3      smallest = curr
4      do :
5          curr = smallest
6          leftChild = 2 * curr + 1
7          rightChild = 2 * curr + 2
8
9          if leftChild < size and A[leftChild] < A[smallest]:
10             smallest = leftChild
11          if rightChild < size and A[rightChild] < A[smallest]:
12             smallest = rightChild
13
```

```

14         if smallest != curr:
15             swap(A[curr], A[smallest])
16         while(smallest != curr)

```

4. MIN-HEAPIFY Complexity Analysis

Our algorithm compares $A[i]$ with its two children, swap if they don't satisfy the Min-Heap RI and then continue the loop with the child index. So the worst case will be: The MIN-HEAPIFY starts with the root, ends at one leaf, visiting all **height** = $\lfloor \log_2 n \rfloor$ levels. In each loop, the three conditional check and one possible swapping all run in $O(1)$. So the worst-case time complexity of MIN-HEAPIFY is $\log_2 n \cdot O(1) = O(\log_2 n)$.

5. Correctness Proof

- Contract of MIN-HEAPIFY :
 - Pc** : $|A| = \text{size} \ \&\& \ 0 \leq i < \text{size}$
 - $\&\&$ the subtree rooted at left child of $A[i]$ is a valid Min-Heap
 - $\&\&$ the subtree rooted at right child of $A[i]$ is a valid Min-Heap
 - Pc'** : $|A| = \text{size} \ \&\& \ 0 \leq i < \text{size}$
 - $\&\&$ the subtree rooted at left child of $A[i]$ is a valid Min-Heap
 - $\&\&$ the subtree rooted at right child of $A[i]$ is a valid Min-Heap
 - $\&\&$ $\text{curr} = i$
 - Inv** : $i \leq \text{curr} < \text{size}$
 - $\&\&$ the subtree rooted at left child of $A[i]$ is a valid Min-Heap
 - $\&\&$ the subtree rooted at right child of $A[i]$ is a valid Min-Heap
 - Qc'** : $A[\text{curr}] < A[\text{leftChild}]$, if it exists $\&\&$ $A[\text{curr}] < A[\text{rightChild}]$, if exist
 - Qc** : the tree rooted at index i is a valid Min-Heap
- Using the Loop Invariant Theorem :
 - Initialization:** Assume **Pc** holds, we need to prove **Inv** hold:
 Before the loop starts, from **Pc'**, we know that $\text{curr} = i$,
 then $i \leq \text{curr} < \text{size}$, the subtree rooted at left child of $A[i]$ is a valid Min-Heap, and the subtree rooted at right child of $A[i]$ is a valid Min-Heap. Then loop invariant holds at the start.
 - Maintenance:** Assume **Inv&&B** holds, we need to prove the loop make **Inv** true again :
Inv&&B : $i \leq \text{curr} < \text{size} \ \&\& \ \text{smallest} \neq \text{curr}$

&& the subtree rooted at left child of $A[i]$ is a valid Min-Heap

&& the subtree rooted at right child of $A[i]$ is a valid Min-Heap

Inside each loop, we compare $A[curr]$ to its two children. If they satisfy the Min-Heap property, then break the loop and `smallest == curr`. Otherwise, swap $A[curr]$ with the smaller child, and continue the loop at `curr = smallest`. So, in any case, whether the value of `curr` be updated or not, `curr` is always smaller than `size` (Because if $2 * curr + 1$ or $2 * curr + 2$ exceed `size`, then they have no chance to be `smallest`). $\Rightarrow$ After the loop, $i \leq curr < size$. And the swap on the level of $A[curr]$ does not influence the subtrees, which are already valid Min-Heaps. Thus, `Inv` still holds after each loop.

- **Termination:** Once the loop ends, `Inv && (not) B` holds, we need to prove `Qc` holds :

`Inv && (not) B` : $i \leq curr < size$ **&&** `smallest == curr`

&& the subtree rooted at left child of $A[i]$ is a valid Min-Heap

&& the subtree rooted at right child of $A[i]$ is a valid Min-Heap

That means no swapping happens in the last loop, the Min-Heap property is eventually satisfied at the last `curr` position. `Inv && (not) B` tells the two subtrees are initially valid Min-Heaps, then after we fix the possible violation, the tree rooted at index i is a valid Min-Heap now. Therefore, `Qc` holds.

Thus, by the **Loop Invariant Theorem**, we've shown our algorithm `MIN-HEAPIFY` is correct.

EXERCISE 2: Min-heap over Dynamic Structure[20 points]

1. Representation Invariant

$RI(R) = \forall \text{ node of } R \ N, N.value \leq N.left.value \ \&\& \ N.value \leq N.right.value \ \&\& \ R \text{ is a almost complete binary tree.}$

Check the files `minHeapRep.java` and `minHeapRepTest.java`.

2. Insertion

```
1  parentOfNew(Node<T> root){
2      if root == null
3          return null
4      Q = new queue()
5      Q.enqueue(root)
6      while(!isEmpty(Q)){
```

```

7         current = Q.dequeue()
8         if(current.left == null or current.right == null)
9             return current
10        Q.enqueue(current.left)
11        Q.enqueue(current.right)
12    }
13 }
14
15 MIN-HEAPIFY(Node<T> node){
16     while(node.parent != null and node.value <= node.parent.value){
17         swap(node.value, node.parent.value)
18         node = node.parent
19     }
20 }
21
22 insertToMinHeap(Node<T> root, T value){
23     newNode = Node<>(value)
24     parent = parentOfNew(root)
25     newNode.parent = parent
26     if(parent == null)
27         root = newNode
28     else if(parent.left == null)
29         parent.left = newNode
30     else if(parent.right == null)
31         parent.right = newNode
32     MIN-HEAPIFY(newNode)
33     return root
34 }

```

3. Complexity of Insertion

Suppose that a min-heap has n nodes.

The complexity of `parentOfNew(Node node)` is $O(n)$ since it traverses almost n nodes from the root in the worst case.

The complexity of `MIN-HEAPIFY(Node node)` is $O(h) = O(\log n)$, where h is the height of the min-heap.

Then the total complexity of `insertToMinHeap(Node root, T value)` is $O(n) = \max(O(n), O(\log n))$.

4. Java Implementation

Check the files `InsertToMinheap.java` and `InsertToMinheapTest.java`.

EXERCISE 3: Sorted Arrays, Balanced and Binary

Search Trees [30 points]

1. Representation & Representation Invariant

Representation : `acBST<A>` = `<A: array>`

Representation Invariant :

`RI` = `A` is sorted(increasing) && `m = length(A)/2`, `A[m]` is the root

&& In the tree representation, levels are filled left to right

&& `A[(m+1)/2]` is the left child of the root

&& `A[:m]` is either empty or can be represented as an `acBST`

&& `A[n-(n-m)/2]` is the right child of the root, `n = length(A)`

&& `A[m+1:]` is either empty or can be represented as an `acBST`

2. acBST is an AVL tree

Recall: In an **AVL** tree, for every node, the heights of the two child subtrees differ by at most one.

- As we said in `RI`, both left and right subtrees of each node are `acBST`;
- In each `acBST`, we figure its root as `length/2`, which means we separate its two subtrees almost-evenly(binarily). More precisely, the number of elements in the left subtree may one more bigger than the one of right subtree;
- Thus, all branches will reach its leaf almost the same time, perhaps one level differ(the branches on the left may one node longer). Each level is filled as evenly as possible, so the difference of height is at most one.

⇒ All `acBST` are AVL trees.

3. Naive AVL Construction Algorithm

(a). Count the Number of Rotations

[code in the wet part : `NaiveAVL.java`]

Output: `Total number of rotations: 7`

(b). Justify

False. 'Almost-complete AVL' requires the last level filled from left to right, as we shown in RI. But the `build(A)` function in the Naive AVL structure may cause the tree to be right-skewed. Thus, not all AVL created by `build(A)` is almost-complete.

4. Rotation-Free AVL Construction

```
1  AVLNode build2(int[] A):
2      n = length(A)
3      if (n == 0) :
4          return null
5      else :
6          m = n / 2
7          AVLNode node = new AVLNode(A[m])
8          node.left = build2(A[:m])
9          node.right = build2(A[m+1:])
10     return node
```

5. Complexity Analysis for the Worst Case

Under this rotation-free strategy, the worst case is the same as its best case(or the general cases). Because the algorithm balances the left and right subtrees recursively, the tree is at most possibly evenly at every step. In any case, each node is created exactly once, and the conditional check repeats $2 \cdot \log_2 n$ times. Thus, the total time complexity is $n \cdot O(1) + 2 \cdot \log_2 n \cdot O(1) = O(n)$.

6. Implementation and Testing

[code in the wet part : `AutoAVL`]

7. Almost-Completeness AVL

Yes, it always generate an almost-complete AVL. Under our definition, the left subtree has `m` elements (from `A[0]` to `A[m-1]`), and the right subtree has `n-m-1` elements (from `A[m+1]` to `A[n-1]`). So the tree we built is either even or left-skewed, with at most one element heavier. Thus, our algorithm always generate an almost-complete AVL.